



Hackathon – Cohort 3



Project 1: Light intensity-based motor speed Control



Goal: Build a closed-loop system that varies motor speed with light intensity.

Bright light → Faster motor Low light → Slower / stop

- **Milestone 1: Measure ambient light intensity using the LDR → ADC .**
 - 1. Interface the LDR with pin P2.1 of the PSoC board. Configure the ADC to measure the voltage across the LDR with a resolution of 12 bits and set it to operate in continuous mode.
 - 2. Continuously monitor the ADC voltage in a loop of the main subroutine
- **Milestone 2: Implement a 100Hz timer functionality that adjusts motor speed based on the light intensity.**
 - 1. Set up a timer module with an interrupt feature to generate a terminal count (TC) interrupt at a frequency of 100 Hz by appropriately configuring the timer clock.
 - 2. Set up a second timer module to generate a 1KHz PWM signal output on pin P1.2
 - 3. Implement a timer interrupt handler function for the 100Hz timer, this ISR monitors the light intensity and adjusts the PWM duty cycle set it to 25% if the ADC reading exceeds 1500 and set it to 75% if the ADC reading goes below 1400. (if necessary, adjust the potentiometer in the LDR board)
- **Milestone 3: Implement an average of last 10 samples for LDR ADC and decide the PWM duty cycles based on average value.**
 - 1. Ensure that the ADC reading retrieved by the 100Hz timer is the average of last 10 readings
 - 2. In the 100Hz Timer interrupt handler, use this average to update the duty of the PWM

Project 2: Auto Street lamp control system - 1 LDRs

Goal: “Smart Street Light” – Make streetlamps bright only when it’s really dark!

– Milestone 1: Configure 9 LEDs on the PSOC board

- 1. Configure 9 GPIOs that are connected to LEDs on the PSOC board to SW controlled GPIO outputs (strong drive)

– Milestone 2: Measure ambient light intensity using the LDR

- 1. Interface the LDR with pin P2.1 of the PSoC board. Configure the ADC to measure the voltage across the LDR with a resolution of 12 bits and set it to operate in continuous mode.
- 2. Continuously monitor the ADC voltage in a loop of the main subroutine

– Milestone 3: Implement a timer interrupt (100 Hz) that adjusts Number of LEDs based on the light intensity

- 1. Set up a timer module with an interrupt feature that triggers a terminal count (TC) interrupt at a frequency of 100Hz by appropriately configuring the clock.
- 2. In the timer interrupt handler function, control the LEDs based on the LDR value as follows:
Turn ON all 9 LEDs if the ADC value exceeds 3000, 6 LEDs if the **ADC value** is greater than 2000 but <3000, 4 LEDs if ADC value is greater than 1500 but <2000, and 2 LEDs if ADC value is greater than 1000 but <1500 and turn OFF all LEDs if the value falls below 1000.
- 3. The **ADC value** mentioned above is the average of last 10 readings.

Project 3: Motor Direction & Speed control with switch

Goal: Change motor rotation direction and RPM (motor speed) on press of a switch

– Milestone 1: Implement Motor speed control using PWM signal

- 1. configure the PWM output on pin P1.2 to generate a 1kHz signal with a duty cycle of 25%.
- 2. Connect the PWM signal to the EN pin of the motor driver. Additionally, use two GPIO pins to control IN1 and IN2, and demonstrate that the motor is rotating.

– Milestone 2: Implement Direction control of motor with GPIO interrupt (toggle switch)

- 1. Configure a GPIO switch on pin P3.7 to trigger a Port 3 interrupt whenever the SW1 switch is pressed. Implement debouncing to ensure reliable operation.
- 2. Once a genuine key press is detected, HALT the motor by driving IN1=IN2=0 and wait for reasonable amount of time using software delays (use your judgement to decide what constitutes reasonable time)
- 3. change the motor's direction after the delay elapses.
- 4. Ensure that the PWM duty cycle is adjusted in increments of 25% as the following:
25% → 50% → 75% → 100% → 25% → ...

Project 4: Washing machine with PSOC



Goal: Alternate the motor rotation direction at every 1sec and 5sec

- Run motor for 5sec -> Halt for 1Sec -> Run the motor in reverse direction for 5 sec → Halt for 1Sec → Run the motor in reverse direction for 5 sec → ..
- **Milestone 1: Setup a timer to generate PWM signal to drive a motor**
 - 1. configure the PWM output on pin P1.2 to generate a 1kHz signal with a duty cycle of 50%.
 - 2. Connect the PWM signal from pin P1.2 to the EN pin of the motor driver. Additionally, use two GPIO pins to control IN1 and IN2, and demonstrate that the motor is rotating.
- **Milestone 2: Setup a second timer to generate 5sec delay**
 - 1. configure a timer module with an interrupt feature to trigger a terminal count (TC) interrupt every 5 seconds
 - 2. In the timer interrupt handler function, implement logic to HALT the motor by driving IN1=IN2=0 and wait for reasonable amount of time using software delays (use your judgement to decide what constitutes reasonable time)
 - Start a third timer that generates an interrupt after 1 second (see next milestone)
- **Milestone 3: Setup a third timer to generate 1sec delay**
 - 1. Configure a timer module with an interrupt feature to trigger a terminal count (TC) interrupt every 1 second by setting up the clock appropriately.
 - 2. After the 1-second timer expires, change the motor's direction of rotation and restart the 5-second timer, creating a looping sequence between the 1-second and 5-second timers.
- **Milestone 4: Show direction of rotation with LED**
 - Glow LED1 when Motor is rotating in clockwise direction, glow LED2 when motor is rotating in anti-clockwise direction

